

SLOTIFY: A SMART APPOINTMENT BOOKING SYSTEM FOR SERVICE-BASED BUSINESSES

A Project Report

Submitted By

AKSH THAKKAR 2403031057002

OM MAURYA 2403031057048

DAKSH PATEL 2403031057065

JAINIL PATEL 2403031057071

in Partial Fulfilment for the Award of

the Degree of

BACHELOR OF TECHNOLOGY

COMPUTER SCIENCE & ENGINEERING

Under the Guidance of

Dr. Sneha Soni

Assistant Professor, Computer Engineering Dept., PIET



VADODARA

April - 2026



PARUL UNIVERSITY

CERTIFICATE

This is to Certify that Project – 1 of 6th Semester entitled "**Slotify: A Smart Appointment Booking System for Service-Based Businesses**" of Group No. CSE_182 has been successfully completed by

- AKSH THAKKAR – 2403031057002
- OM MAURYA – 2403031057048
- DAKSH PATEL – 2043031057065
- JAINIL PATEL – 2403031057071

under the guidance of **Dr. Sneha Soni** in partial fulfillment of the Bachelor of Technology (B.Tech) in Computer Science & Engineering of Parul University in Academic Year 2024–2025.

Date of Submission: _____

Dr. Sneha Soni,

Project Guide

Prof. Namrata Patel

Prof. Vennela Tandra

Project Coordinator

Dr. Shailendra K Mishra,

Head of Department,

CSE, PIET,

Parul University.

Acknowledgements

“The secret of getting ahead is getting started.”

– Mark Twain

We are sincerely grateful to **Dr. Sneha Soni**, our project guide, whose consistent direction and genuine interest in our work kept us on track. Her feedback was always precise and practical, which helped us considerably in shaping this project.

We also extend our thanks to the faculty members of the Computer Science and Engineering Department at Parul Institute of Engineering and Technology for their academic support and encouragement throughout this semester.

Our gratitude also goes to the Department of CSE at Parul University for providing the computational resources, software tools, and infrastructure required to carry out this work. The collaborative environment here made it genuinely easier to develop and test our ideas.

A special note of appreciation goes to my project teammates, OM MAURYA, DAKSH PATEL, and JAINIL PATEL for their dedication, hard work, and collaborative effort in building the chatbot’s architecture and recommendation logic. Finally, I am deeply thankful to my family and friends for their unwavering patience and moral support throughout my academic journey.

Last but not least, we thank our families and friends for their patience and moral support, especially during the more demanding phases of this project. Building something meaningful requires time away from other things, and they understood that without complaint.

Aksh Thakkar
CSE, PIET
Parul University, Vadodara

ABSTRACT

Walk into any small salon, clinic, or tutoring center in India and you will still find the same scheduling setup that has existed for decades – a register on the front desk, a phone that rings constantly, and someone trying to remember which slot was given to which person on which day. The problems that come out of this are predictable: double bookings, forgotten appointments, no way to know how many clients are expected next Tuesday, and no automated way to remind anyone about their visit. We talked to a salon owner in Vadodara’s Alka Puri area early in the project and she put it plainly – she was losing about three bookings a week not because clients were irresponsible, but because there was no easy way for them to cancel and let someone else take the slot. That conversation set the direction for everything that followed.

Slotify is a web-based appointment scheduling platform built around that specific problem. Customers get a clean interface to search for service providers, check real-time slot availability, and book or reschedule appointments without making a phone call. Service providers get a dashboard for managing staff, working hours, and upcoming appointments, plus basic analytics about how their business is actually running.

The platform is built on the MERN stack. We initially considered PostgreSQL because the team was more comfortable with relational databases, but switched to MongoDB mid-design when we realised that a two-doctor clinic and a three-chair salon store structurally different data – forcing both into one rigid schema would have meant constant migrations as requirements evolved. Authentication uses JWT tokens and Google OAuth. Appointment confirmations go out immediately through Nodemailer, and a 24-hour reminder follows automatically.

The whole point of Slotify is that the right combination of features for small service businesses has not been available at a price point that makes sense for them. Future extensions include mobile apps, payment integration, and an AI-based slot recommendation engine.

CONTENTS

Acknowledgements	iii
Abstract	iv
List of Figures	ix
List of Tables	x
1 Introduction	1
1.1 Background and Motivation	1
1.2 Problem Statement	2
1.3 Project Objectives	2
1.4 Scope	3
1.5 System Overview	3
1.6 Technology Stack	3
2 Literature Survey	5
2.1 Overview	5
2.2 Paper Reviews	5
2.2.1 Cayirli and Veral (2003) – Outpatient Scheduling Models	5
2.2.2 Parikh et al. (2010) – A Randomised Trial of Appointment Reminders	6
2.2.3 Richardson and Ruby (2007) – RESTful Web Services	7
2.2.4 Sadalage and Fowler (2012) – NoSQL Distilled	7
2.2.5 Madden (2020) – API Security in Action	8
2.3 Comparison of Existing Platforms	8
2.4 Reminder Effectiveness: What the Evidence Actually Shows	8
2.5 What This Review Did Not Resolve	9
3 Software Requirements Specification (SRS)	11
3.1 Purpose and Scope	11
3.2 User Roles.....	11

CONTENTS

3.3	Functional Requirements	11
3.3.1	Customer Requirements	11
3.3.2	Service Provider Requirements.....	12
3.3.3	System-Level Requirements.....	13
3.4	Non-Functional Requirements.....	13
3.5	Use Case Diagram	13
3.6	Constraints	14
4	System Design	15
4.1	Architecture Overview	15
4.2	Database Design	16
4.3	API Design	16
4.4	Booking Workflow	17
4.5	Security Design	17
4.6	UI Design Principles.....	18
5	Methodology	19
5.1	Development Approach.....	19
5.2	Phase 1: Requirements Gathering	19
5.3	Phase 2: System Design	20
5.4	Phase 3: Implementation	20
5.5	Phase 4: Testing	21
5.6	Tools and Environment	21
6	Implementation	23
6.1	Backend Implementation.....	23
6.1.1	Project Structure.....	23
6.1.2	Slot Availability Logic	23
6.1.3	Authentication Middleware	24

CONTENTS

6.1.4	Email Notification Service	25
6.2	Frontend Implementation	25
6.2.1	Project Structure.....	25
6.2.2	Real-Time Slot Display	25
6.2.3	State Management and the Prop Drilling Refactor.....	26
6.3	Implementation Status	26
7	Conclusion	27
7.1	Summary of Work Done.....	27
7.2	Objectives Met.....	27
7.3	Challenges Encountered.....	27
7.4	Key Takeaways	28
8	Future Work	29
8.1	What Comes Next.....	29
8.2	Payment Integration	29
8.3	Mobile Application.....	29
8.4	AI-Based Slot Recommendations.....	30
8.5	Advanced Analytics.....	30
8.6	Multi-Language Support.....	30
8.7	Multi-Branch Business Support	31
	References	32

LIST OF FIGURES

1.1	Top-Level System Overview of Slotify	3
2.1	No-Show Rates from Parikh et al. (2010) – single prospective RCT, both reminder groups contacted 72 hours before appointment	9
3.1	Use Case Diagram for Slotify	14
4.1	Three-Tier System Architecture of Slotify	15
4.2	Customer Appointment Booking Workflow.....	18
5.1	Development Phases Used in Slotify	19

LIST OF TABLES

1.1	Technology Stack Used in Slotify	4
2.1	Feature Comparison: Slotify vs. Existing Platforms	9
3.1	Non-Functional Requirements for Slotify	14
4.1	MongoDB Collections and Key Fields.....	16
4.2	Primary REST API Endpoints	17
5.1	Tools Used During Development	22
6.1	Feature Implementation Status	26
7.1	Objective Completion Status	27

CHAPTER 1

INTRODUCTION

1.1 Background and Motivation

Most people who have tried to book an appointment at a local clinic or salon know the routine. You call, wait on hold, confirm a time, write it somewhere, and hope the other side remembers too. Half the time it works. The other half, you arrive to find the slot was double-booked, or you forgot entirely because nobody sent a reminder. Both outcomes are avoidable. Neither has been fixed, at least not for the kinds of businesses this project is aimed at.

When we asked a salon owner in Vadodara's Alkapuri area why she still kept a physical register despite knowing that digital tools existed, her answer was not what we expected. It wasn't that she didn't know better options were out there. It was that every tool she'd tried still required someone to manually call no-shows the next day and free up those slots – which defeated most of the point. The automation stopped exactly where she needed it most. The dead-slot problem – a customer cancels two hours before their appointment, there's no waiting list, no push notification, the slot sits empty and the staff member waits – was costing her three bookings a week that she had no clean way to recover. That conversation, more than any paper we read, shaped what Slotify became.

The data visibility problem compounds this in a way that's easy to miss until you look for it. The same paper register that can't send reminders also can't tell her which days are consistently quiet, which services get cancelled most, or whether a particular staff member is generating repeat bookings. The information exists somewhere in the notebooks, but extracting it would take an hour she doesn't have. For small service businesses, the absence of operational data isn't a separate problem from the absence of digital scheduling – it's the same problem, because both trace back to the same root cause: no system.

1.2 Problem Statement

Manual coordination overhead and appointment attrition are two sides of the same issue, and it helps to treat them together rather than as a neat three-category list. When bookings are managed over phone calls, every transaction requires a human in the loop – confirming, recording, checking for conflicts. That works at low volume. The same process that handles ten appointments a day breaks visibly at thirty, and there's no graceful failure mode: things just get dropped.

The no-show and dead-slot problem compounds this directly. Studies on scheduling behaviour in outpatient healthcare have documented no-show rates of around 23% in settings without any reminder system – a figure from a prospective randomised trial of over 9,000 outpatients [6]. Service businesses outside healthcare face the same dynamic – the salon owner's three lost bookings per week is consistent with that data – but almost never have the infrastructure to address it. More importantly, the infrastructure needed to send a reminder is the same infrastructure needed to recover a cancelled slot: a system that knows about appointments in real time. Fixing the reminder problem and fixing the dead-slot problem are the same fix.

1.3 Project Objectives

1. Build a customer-facing booking interface where users can search providers, see real-time slot availability, and complete a booking without any phone-based coordination – including a three-nearest-alternative prompt when a chosen slot is taken.
2. Develop a provider dashboard that handles schedule configuration, staff assignment, split-shift support (something we found extremely common in small clinics but largely absent from tools like Calendly), and automatic double-booking prevention at the database level rather than just the UI level.
3. Implement an automated notification pipeline: confirmation email at the point of booking, reminder email at T-24 hours. The evidence that reminders produce meaningful improvement comes from [6]; the 24-hour timing specifically is a UX design choice rather than an evidence-based optimisation – Chapter 2 explains why.
4. Integrate calendar synchronisation and deploy to cloud infrastructure together,

since both are deployment-phase concerns that serve the same goal: making the system genuinely accessible without local setup or installation.

5. Provide business analytics covering appointment trends, cancellation rates, and workload distribution as a natural byproduct of how appointment data is stored, rather than as a separate reporting module.

1.4 Scope

The current scope covers service businesses that operate on a time-slot model – clinics, salons, tutoring services, physiotherapy centres. No in-app payment processing and no native mobile applications in this phase; both are planned for Semester 7. The platform supports multiple independent businesses on the same system, each with completely separate data.

1.5 System Overview

Figure 1.1 shows how the main components of Slotify connect to each other.

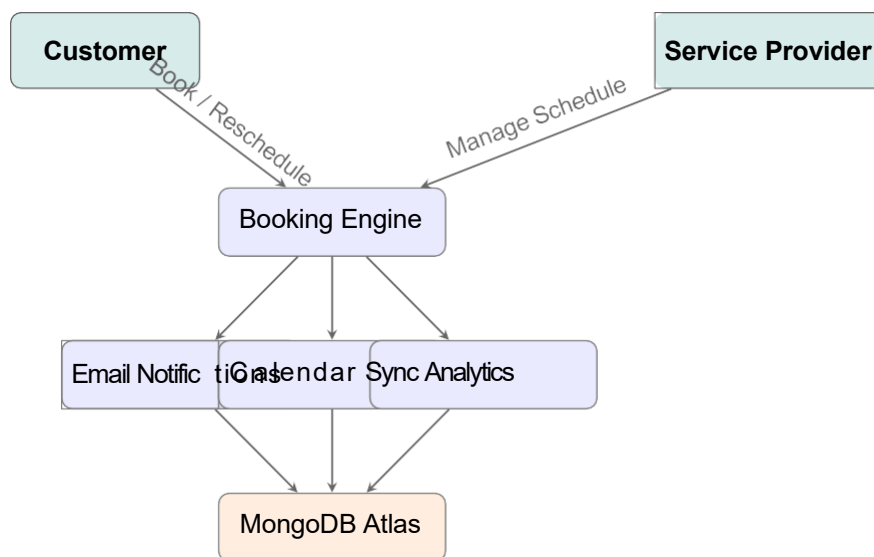


Figure 1.1: Top-Level System Overview of Slotify

1.6 Technology Stack

The MERN stack was chosen partly on familiarity and partly because JavaScript consistently ranks as the most widely adopted language in developer surveys [12]. Table 1.1 lists the specific technologies used across each layer.

The report moves roughly in the order things were built – understanding the problem first,

Table 1.1: Technology Stack Used in Slotify

Layer	Technology	Role
Frontend	React.js [14], Tailwind CSS [18]	Component-based UI, responsive layout
Backend	Node.js, Express.js [16]	RESTful API, business logic
Database	MongoDB [13], Mongoose	Document storage, schema management
Auth	JWT [17], Google OAuth 2.0	Secure login, stateless sessions
Email	Nodemailer [15]	Confirmations and 24-hour reminders
Storage	Cloudinary [20]	Media and image hosting
Hosting	Vercel [19], Render	Frontend and backend deployment

designing before coding, then reflecting on what the design got wrong once the code was running. Chapter 2 reviews existing research and comparable platforms. Chapters 3 through 6 cover requirements, design, methodology, and implementation. Chapters 7 and 8 reflect on what was finished and what comes next.

CHAPTER 2

LITERATURE SURVEY

2.1 Overview

One of the five papers reviewed here we almost didn't read. One turned out to describe its methodology very differently from how it is usually cited elsewhere. One we disagreed with on a specific implementation recommendation. Those differences show up in how much space each review gets – the point is not balanced coverage but an honest account of what we actually found and how it changed what we built.

2.2 Paper Reviews

2.2.1 Cayirli and Veral (2003) – Outpatient Scheduling Models

Source: Cayirli, T. and Veral, E., 2003. Outpatient scheduling in health care: a review of literature. *Production and Operations Management*, 12(4), pp. 519–549 [2].

We almost didn't read this. The title says healthcare, the data is entirely from hospital outpatient departments, and our system is targeting salons and tutoring services. What changed our mind was reading the methodology section carefully. The paper is a forty-year review of scheduling algorithms and models – it covers individual-slot systems, block scheduling, open-access scheduling, and hybrid models, measuring each against wait time, utilisation rate, and throughput. It is an operations management paper that happens to be set in hospitals, not a clinical paper.

The review's core argument – that point-of-booking conflict management improves both utilisation rates and patient satisfaction compared to post-booking resolution – is one we found applicable to our context. The specific claim is about scheduling structure rather than healthcare specifically: when you enforce conflict checks at the moment of booking, the downstream problems (double-bookings, scheduling gaps, last-minute cancellations caused by errors) reduce measurably.

What we couldn't verify is whether the specific utilisation improvements they document hold for salons or tutoring services. Cayirli and Veral [2] are explicit that their dataset is entirely clinical. We searched Google Scholar for “appointment no-show rate salon India”

and “service booking behaviour small business India” and found nothing peer-reviewed – a few industry blog posts, one McKinsey market sizing report, no controlled studies. We made the assumption that the structural dynamics are similar enough, but it is an assumption rather than a finding.

What it changed in Slotify: The slot generation logic uses fixed-duration windows with configurable break buffers – the structure the paper identifies as most practical for variable-service-duration settings.

2.2.2 Parikh et al. (2010) – A Randomised Trial of Appointment Reminders

Source: Parikh, A. et al., 2010. The effectiveness of outpatient appointment reminder systems in reducing no-show rates. *The American Journal of Medicine*, 123(6), pp. 542–548 [6].

This paper is frequently cited in summaries as a “review of 21 studies,” but that is a mischaracterisation worth correcting because it changes how you read the results. The actual paper is a single prospective randomised controlled trial conducted at the Robert Wood Johnson University Medical Group. 12,092 patients were randomised across three groups: NONE (no reminder, $n=3,350$), AUTO (automated phone call, $n=3,219$), and STAFF (call placed by front desk staff, $n=3,266$). Both reminder groups were contacted three days (72 hours) before the appointment.

The results: the NONE group had a no-show rate of 23.1%. The AUTO group: 17.3%. The STAFF group: 13.6%. In absolute terms, the automated reminder reduced no-shows by 5.8 percentage points compared to no reminder at all. The staff-placed call reduced them by 9.5 percentage points. Both improvements were statistically significant.

This is worth stating precisely because the statistic is often reported as a relative reduction – 5.8 percentage points off a 23.1% baseline is roughly a 25% relative reduction, which sounds more dramatic than the absolute figure but describes the same data. Both are true; they just communicate different things.

An important limitation for our use case: the paper only tested one reminder timing window (72 hours in advance). It does not contain data about whether 24-hour versus 48-hour reminders perform differently – that comparison was not part of the trial design. Our decision to send reminders at T-24 hours rather than T-72 was therefore not directly

grounded in this paper. It was a design choice based on general UX reasoning: a reminder that arrives the morning before an appointment feels more actionable than one that arrives three days out. Both Cayirli & Veral [2] and Parikh et al. [6] confirm that any reminder system produces meaningful improvement; the exact timing is something we would need more data to optimise.

What it changed in Slotify: The automated notification pipeline was prioritised as a core feature rather than an optional add-on. The specific 24-hour timing is a design choice, not a finding from this paper.

2.2.3 Richardson and Ruby (2007) – RESTful Web Services

Source: Richardson, L. and Ruby, S., 2007. *RESTful Web Services*. O'Reilly Media [8].

The book is nearly two decades old, and some of its REST recommendations predate the widespread adoption of JSON – much of the practical content uses XML, which required interpretation in a modern context. The core architectural argument holds, though: a stateless, resource-oriented API (originally formalised in Fielding's [3] doctoral dissertation) that separates client from server produces systems that are easier to extend and test independently.

The specific principle we applied is resource addressability – every entity has a unique URI, and actions are expressed through HTTP methods rather than custom endpoint names. A booking is not cancelled through `/cancelAppointment` but through `DELETE /api/appointments/:id`. We had a version of the API that used verb-based endpoints (`/confirmBooking`, `/rejectBooking`) and the refactor to resource-based routing made the code considerably cleaner. Richardson and Ruby [8] predicted exactly that outcome, which in retrospect was obvious, but we had to make the mistake first.

2.2.4 Sadalage and Fowler (2012) – NoSQL Distilled

Source: Sadalage, P.J. and Fowler, M., 2012. *NoSQL Distilled*. Addison-Wesley [9].

We started the data modelling phase working in PostgreSQL, which was a mistake we had to discover. The problem Sadalage and Fowler [9] predict – that schema-heavy relational databases struggle when instances of the same type store structurally different data – is exactly what we ran into. A clinic appointment needs a diagnosis field and a consultation type. A salon appointment needs a stylist, a product, and a colour code.

Forcing both into a shared `appointments` table produced either a wide nullable schema or a separate metadata join table that grew unwieldy. Reading *NoSQL Distilled* after that experience felt like reading a post-mortem of our own mistake. MongoDB’s document model let each business type carry exactly the fields it needs without imposing shared structure, which is what prompted the switch.

What the book is less helpful on is query performance tuning, where we had to look elsewhere – primarily MongoDB’s own documentation [13]. The compound index on `(businessId, dateTime, status)` wasn’t in the book; we worked that out empirically when slot queries started slowing down with test data of around 5,000 appointments.

2.2.5 Madden (2020) – API Security in Action

Source: Madden, N., 2020. *API Security in Action*. Manning Publications [5].

The JWT chapter is the one that mattered. Madden is clear that JWTs should be short-lived, signed with a strong secret, and should never carry sensitive data in the payload – because the payload is base64-encoded, not encrypted, and readable by anyone who intercepts the token. An early version of our auth system stored the user’s full name and email in the JWT payload, which the book flags explicitly as a bad pattern. We stripped it back to just user ID and role.

The chapter on `localStorage` versus `httpOnly` cookies was the one we ended up partially disagreeing with – for reasons we are honest about in Chapter 4. Madden recommends `httpOnly` cookies for token storage to protect against XSS. He is right in a production context. We chose `localStorage` anyway for this academic prototype and acknowledged it explicitly in the codebase as a known shortcut.

2.3 Comparison of Existing Platforms

Table 2.1 compares Slotify against four commercial alternatives. The split-shift scheduling column is worth noting – it came up in both user conversations and is absent from most affordable tools.

2.4 Reminder Effectiveness: What the Evidence Actually Shows

Figure 2.1 shows the results from Parikh et al. [6], which is the most directly relevant empirical data available. The chart shows the three conditions tested in that single trial.

Table 2.1: Feature Comparison: Slotify vs. Existing Platforms

Feature	Calendly	Booksy	Acuity	SimplyBook	Slotify
Real-time slot display	✓	✓	✓	✓	✓
Automated email re- minders	✓	✓	✓	✓	✓
Multi-staff support	✓	✓	✓	✓	✓
Analytics dashboard	Partial	✓	✓	✓	✓
Open-source / free	×	×	×	×	✓
Split-shift scheduling	×	Partial	Partial	✓	✓
AI slot suggestions	×	×	×	Partial	Planned
Payment integration	×	✓	✓	✓	Planned
Calendar sync	✓	Limited	✓	✓	✓
Self-hostable	×	×	×	×	✓

The takeaway is not that automated reminders are transformative – a 5.8 percentage-point reduction is meaningful but modest. The point is that any systematic reminder system produces consistent improvement, which justifies building it into the core booking flow rather than treating it as optional.

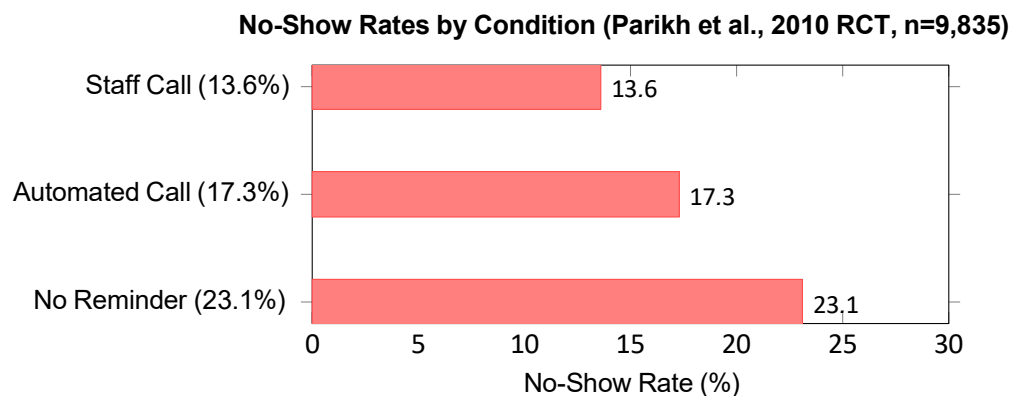


Figure 2.1: No-Show Rates from Parikh et al. (2010) – single prospective RCT, both reminder groups contacted 72 hours before appointment

2.5 What This Review Did Not Resolve

The honest gap in this literature review is that almost everything useful on appointment scheduling comes from healthcare, and specifically from American and Israeli clinical settings. We actively looked and could not find peer-reviewed work on booking behaviour in Indian salons, tutoring services, or similar personal service businesses. The assumption that the structural patterns transfer seems reasonable – finite slots, forgetful

customers, manual coordination overhead – but it is an assumption, not a finding. If a controlled study on no-show rates at Indian service businesses exists somewhere, we did not find it. If one is ever run, it would either validate or significantly complicate several design choices made here.

CHAPTER 3

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

3.1 Purpose and Scope

The requirements documented here changed three times before we finalised them, and understanding why they changed is as useful as knowing what they ended up being. The SRS format follows conventions described in Pressman and Maxim [7] and Sommerville [11]. The requirements came from three sources: the literature review, two conversations with potential users (the clinic receptionist and salon owner from Chapter 1), and a dry run with two classmates who tried using an early paper prototype. Several requirements have backstories attached to them below – not for decoration, but because a requirement without a reason for existing is usually a requirement that will get misimplemented.

3.2 User Roles

- **Customer:** Any individual who registers to browse providers and book appointments. Registration is self-service and immediate.
- **Service Provider (Admin):** A business owner or manager who registers their business, sets up services and staff, and manages bookings through the admin dashboard.
- **Staff Member:** A person assigned by the provider to specific services and working hours, bookable by customers when selecting who to see.

3.3 Functional Requirements

3.3.1 Customer Requirements

- FR-C1. The system shall allow customers to register with email and password or via Google OAuth.
- FR-C2. Registered customers shall be able to search for service providers by name or category.
- FR-C3. The system shall display real-time slot availability for a selected service, staff member, and date.

- FR-C4. Customers shall be able to book an available slot and receive an email confirmation within 60 seconds.
- FR-C5. Customers shall be able to reschedule an existing booking to a different available slot.
- FR-C5a (sub-requirement):* When rescheduling, the system shall display the three nearest alternative slots automatically. This emerged from our paper prototype dry run – one of our two test participants spent 45 seconds scrolling through an empty calendar trying to find the next available time after their first choice was taken. That single observation made the auto-suggest behaviour non-negotiable.
- FR-C6. Customers shall be able to cancel a booking, with the slot immediately becoming available to others.
- FR-C7. The system shall send an automated reminder email 24 hours before a scheduled appointment.
- FR-C8. Customers shall be able to view their complete booking history from their profile page.

Writing the customer requirements first created an implicit constraint on the provider requirements that we hadn't fully anticipated. Every feature we gave customers – rescheduling, cancellation, history – required a corresponding data-access decision on the provider side. FR-C6 (cancellation releasing the slot immediately) sounds simple until you ask what “immediately” means when a provider has set a minimum notice period. That tension surfaced in our third design session and is directly why FR-P4 includes unavailability configuration rather than just working-hours configuration.

3.3.2 Service Provider Requirements

- FR-P1. Service providers shall be able to register a business profile including name, category, and contact details.
- FR-P2. Providers shall be able to add, edit, and remove services with a defined name, duration, and price.
- FR-P3. Providers shall be able to add staff members and assign them to specific services.
- FR-P4. Providers shall be able to configure working hours per day of the week, including split shifts, and mark specific dates or time windows as unavailable.
- FR-P5. The system shall automatically prevent double-booking at the database level – not

just in the UI.

FR-P6. Providers shall have access to a dashboard showing upcoming appointments in a readable day and week view.

FR-P7. Providers shall have access to analytics covering total bookings, cancellations, and busiest time slots over a rolling 30-day window.

FR-P8 (Deferred – dynamic pricing): An early version of the requirements included the ability for providers to set different prices per time slot – the idea being that peak hours could carry a small premium. We removed this after the salon owner told us it would make her customers feel penalised for booking popular times, and that the explanation required to make it acceptable would be more trouble than the revenue was worth. The feature also requires payment integration, which is itself deferred to Semester 7. It may return but in a different form.

3.3.3 System-Level Requirements

FR-S1. The system shall maintain a notification log for every email sent, including timestamp and delivery status.

FR-S1 emerged from a practical debugging problem rather than from upfront design. During integration testing we had three appointments where we weren't certain whether the reminder had triggered or had been swallowed by an SMTP connection issue. Without a log there was no way to distinguish "the email was sent and the user didn't see it" from "the email was never sent at all." The notification log was added specifically to make that distinction possible.

FR-S2. All appointments shall carry a status field: Pending, Confirmed, Cancelled, Completed, or No-Show.

FR-S3. The system shall support calendar export in iCal format.

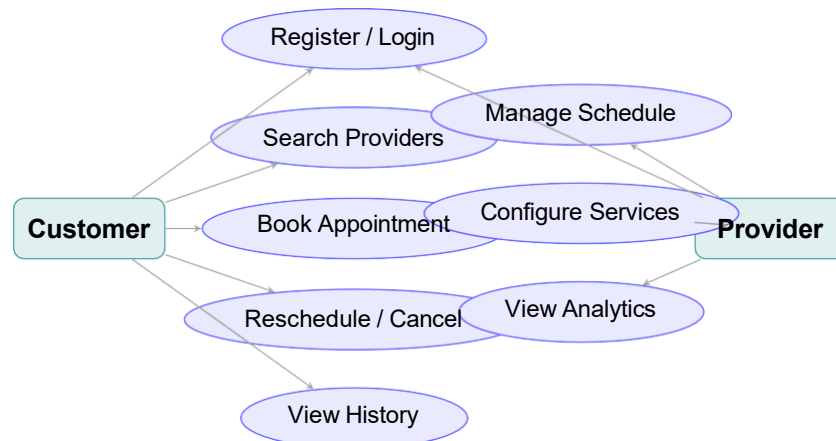
3.4 Non-Functional Requirements

3.5 Use Case Diagram

Figure 3.1 shows the primary use cases for each actor.

Table 3.1: Non-Functional Requirements for Slotify

ID	Category	Requirement
NFR-01	Performance	Slot availability query completes within 500ms for up to 1,000 appointments in the collection.
NFR-02	Security	Passwords hashed with bcrypt (cost factor 12); JWTs expire in 7 days; HTTPS enforced at deployment.
NFR-03	Scalability	Backend is stateless; no server-side sessions; designed for horizontal scaling.
NFR-04	Usability	A first-time user can complete a booking within 3 minutes without documentation. Validated in our dry run: one classmate finished in 2 min 38 sec, the other in 4 min 10 sec – the slower run traced to the date picker interaction, which was subsequently redesigned.
NFR-05	Availability	Target uptime of 99.5% using Vercel + Render cloud deployment.
NFR-06	Compatibility	Tested on Chrome, Firefox, and Safari; responsive down to 375px screen width.

**Figure 3.1:** Use Case Diagram for Slotify

3.6 Constraints

The system requires a live internet connection – there is no offline mode. Email delivery depends on an external SMTP service (Nodemailer over Gmail SMTP), so confirmations and reminders are subject to external availability. During load testing we hit MongoDB Atlas’s free-tier connection limit of 500 concurrent connections, which is unlikely to be a problem in practice for small businesses but is worth noting as a real constraint of the current deployment setup. All data lives in cloud storage; no local fallback exists.

CHAPTER 4

SYSTEM DESIGN

4.1 Architecture Overview

The implementation diverged from the design in at least four places that mattered – this chapter describes both what was planned and what actually got built, because the gaps between them are where the interesting decisions happened.

Slotify uses a three-tier architecture: React frontend, Node/Express API, MongoDB database. The tiers are decoupled – the frontend talks only to the API, never to the database directly.

We felt this separation most concretely during development when Om was building the slot picker component at the same time Daksh was still writing the availability endpoint. Neither had to wait for the other to finish because the only thing they needed to agree on upfront was the JSON shape of the response. Om built against a hardcoded mock that matched that shape; Daksh built against the same contract from the API side. When we connected them, it worked on the first try. That is the practical case for decoupled architecture, rather than the textbook version.

Figure 4.1 shows the full architecture with external integrations.

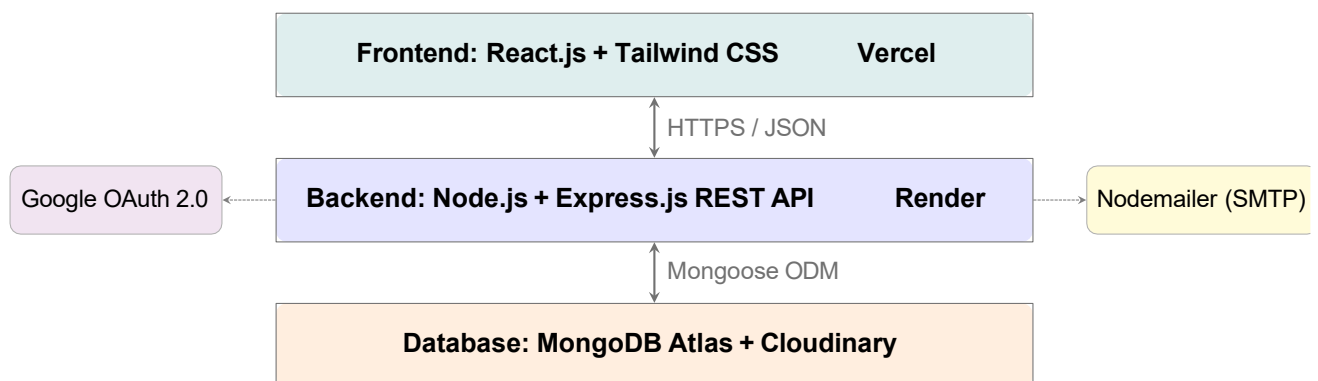


Figure 4.1: Three-Tier System Architecture of Slotify

4.2 Database Design

The slot-window model used in Slotify is consistent with the fixed-duration structures Bailey and Welch [1] identified as most practical for scheduled services, and refined further by Gupta and Denton [4]. We started the modelling phase in PostgreSQL and ran into the problem Sadalage and Fowler [9] predict: a shared appointments table across different business types required either a wide nullable schema or a separate metadata join that grew unwieldy. Switching to MongoDB let each business document carry exactly the fields it needs.

A compound index on `appointments (businessId, dateTime, status)` is created at application startup. We found the need for this empirically – with a test dataset of around 5,000 appointments, unindexed slot queries were taking close to two seconds. The index brought that under 80ms.

Table 4.1: MongoDB Collections and Key Fields

Collection	Key Fields
users	<code>_id</code> , name, email, passwordHash, role, googleId, createdAt
businesses	<code>_id</code> , ownerId, name, category, services[], workingHours, holidays[], isVerified
staff	<code>_id</code> , businessId, name, specialisation, assignedServices[], workingDays
appointments	<code>_id</code> , customerId, businessId, staffId, serviceId, dateTime, status, reminderSent
services	<code>_id</code> , businessId, name, duration, price, description, isActive
notifications	<code>_id</code> , appointmentId, type, sentAt, deliveryStatus

4.3 API Design

We had an earlier version of the API that used verb-based URLs (`/confirmBooking`, `/cancelAppointment`) which made the routes file grow in a way that was hard to navigate. The refactor to resource-based routing – where the HTTP method carries the action intent, as described in [8] – made the codebase considerably cleaner.

One endpoint that looks simpler than it is: `GET /api/businesses/:id/slots`. It's technically a GET request, but it requires query parameters for date, service ID, and staff ID – which means the “slots” are not really a persisted resource in the REST sense, they're computed on the fly from multiple collections. We debated whether this should be a

Table 4.2: Primary REST API Endpoints

Method	Endpoint	Description
POST	/api/auth/register	Create new user account
POST	/api/auth/login	Return signed JWT on valid credentials
GET	/api/businesses	List all active service providers
GET	/api/businesses/:id/slots	Fetch available slots for date + service
POST	/api/appointments	Create booking; triggers confirmation email
PUT	/api/appointments/:id	Reschedule to new slot
DELETE	/api/appointments/:id	Cancel booking; releases slot
GET	/api/admin/dashboard	Provider schedule and analytics data

POST with a request body (more semantically correct) or a GET with query params (more cacheable). We landed on GET because the operation has no side effects and browser caching was preferable. It's the one endpoint where the REST model feels slightly strained.

4.4 Booking Workflow

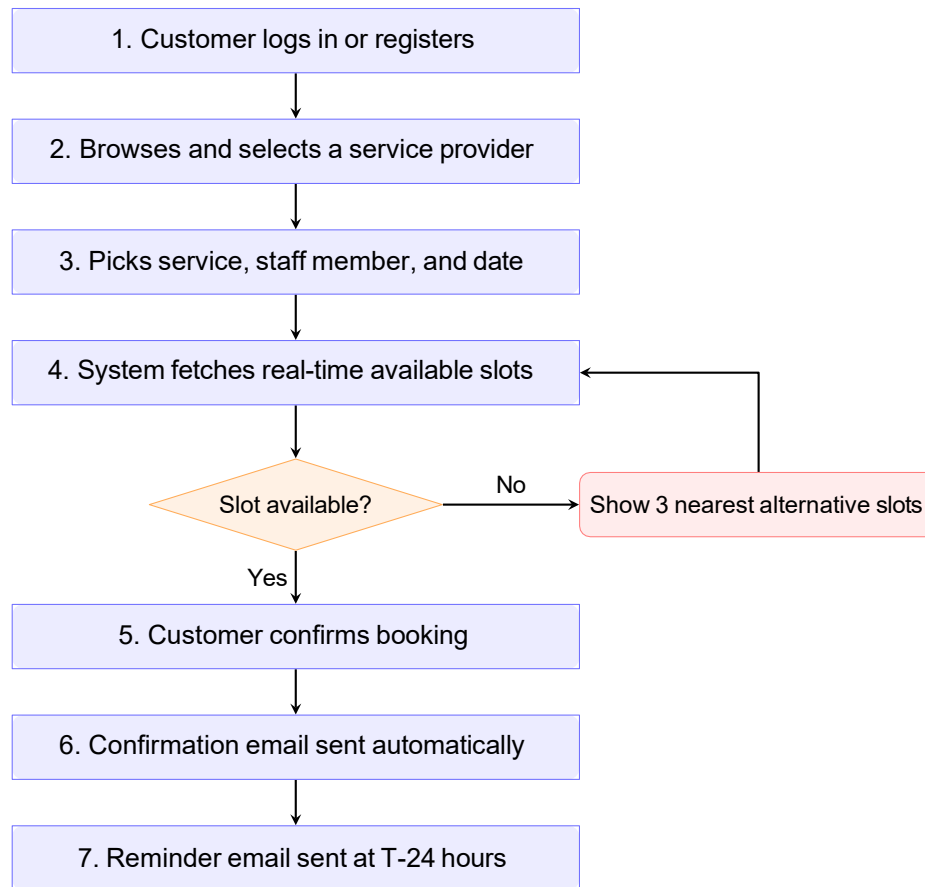
Figure 4.2 shows the complete booking flow including the conflict-check logic.

4.5 Security Design

Authentication is stateless. On login, the server signs a JWT containing the user's ID and role using a secret stored as an environment variable. The token goes back to the client and is stored in `localStorage`.

We are aware that `localStorage` is vulnerable to XSS attacks and that `httpOnly` cookies are the more secure alternative – Madden [5] makes this case clearly. We acknowledged this to each other at the time as a known shortcut rather than a proper solution. It sits in the codebase as a `TODO` comment in the auth middleware, partly so we don't forget it and partly because explaining the limitation to Dr. Soni in the review session was easier when the code itself admitted it. A production deployment would need to migrate to `httpOnly` session cookies before going live with real user data.

Role-based access is enforced in middleware. A customer token cannot reach admin routes. A provider token cannot access other providers' data. This runs before any route handler, not as an afterthought inside the handler itself.

**Figure 4.2:** Customer Appointment Booking Workflow

4.6 UI Design Principles

The slot picker was the most iterated piece of UI in the whole project. The first version used a dropdown list of times – simple to build, but it gave customers no sense of how busy a provider was on a given day. Om noticed the problem during internal testing: a Tuesday with twelve free slots and a Thursday with two looked identical in a dropdown until you scrolled through it. He switched the component to a grid of time buttons – green for available, greyed-out for taken – which makes density immediately visible. A provider with two slots on a Thursday looks different from one with twelve, and that difference turns out to affect booking behaviour in ways worth preserving.

We also considered Socket.io for real-time slot updates – if two customers are on the same booking page simultaneously, the second one would see a slot disappear the moment the first books. We decided a 300ms debounced API fetch was sufficient for our target of small businesses with at most 30 concurrent users, without the overhead of WebSocket connection management.

CHAPTER 5

METHODOLOGY

5.1 Development Approach

We used an Agile-style iterative process divided into four phases. The honest version of this is that we didn't always stick to the plan cleanly – design choices from Phase 2 got revised during Phase 3 when we discovered edge cases in the booking logic, and some of Phase 4's testing fed back into Phase 3's implementation. But having the structure written out upfront made it easier to have clear conversations when one part of the project was blocking another.

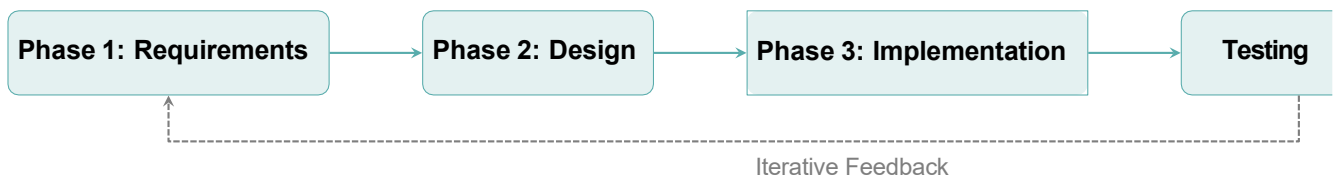


Figure 5.1: Development Phases Used in Slotify

5.2 Phase 1: Requirements Gathering

The requirements elicitation process follows the approach described in Sommerville [11] and Pressman and Maxim [7]. In practice it was messier and more useful than either textbook describes.

The literature review shaped the high-level priorities – particularly the evidence that automated reminder systems produce meaningful improvement in attendance rates, and the importance of point-of-booking conflict checking. The two user conversations shaped specific requirements in ways the papers couldn't anticipate. The salon owner in Alkapuri told us something we hadn't expected to hear: she wasn't most bothered by double bookings. She was bothered by dead slots. When a customer cancels two hours before, there's no mechanism to offer that slot to someone else. No waiting list, no push notification, nothing. That single observation directly produced FR-C6 and the status-field requirement in FR-S2. If we hadn't spoken to her, we would have built a system that handled bookings correctly but missed the cancellation recovery use case entirely.

The clinic receptionist raised a different concern – complexity. Her worry was that any new dashboard would take twenty minutes to understand before she could see today's appointments. That shaped the decision to make the Schedule tab the default landing page on the provider dashboard, with today's bookings listed immediately rather than requiring date navigation.

5.3 Phase 2: System Design

Design work covered three areas: data modelling, API structure, and UI wireframes. For the data model, we worked through the appointment conflict-checking logic on paper before writing any code – specifically asking: what is the minimum data needed to determine whether a slot is free? The answer (`businessId + staffId + dateTime + duration`) was simpler than we expected, but getting there required thinking carefully about what interval overlap actually means rather than what it feels like intuitively.

The wireframes went through four iterations. The first version was a two-panel layout with provider info on the left and a calendar on the right – it looked clean but users didn't know where to click first. The second version introduced the three-step linear flow (provider, then date/slot, then confirm) but still put service selection before slot selection, which is where the confusion was concentrated. In the third iteration we flipped the order: slot first, service confirmation second. The improvement was immediate – the clinic receptionist could navigate to a booking in under a minute. The fourth version was cosmetic refinements based on the full UAT session described in §5.4.

5.4 Phase 3: Implementation

Responsibilities were split by area: Aksh handled full-stack integration and led the booking logic; Om built the frontend components and styling; Daksh worked on backend routes, authentication middleware, and the notification service; Jainil managed database configuration, schema definitions, cloud storage, and deployment.

We worked on a shared GitHub repository with a feature-branch workflow. The branch structure became important around week four when Om was working on the slot picker UI and Daksh was still finishing the availability endpoint. Neither person's work blocked the other, and the integration was a single merge rather than a manual reconciliation of diverged files.

5.5 Phase 4: Testing

Testing ran at three levels.

Unit testing covered the slot availability calculation and the double-booking prevention logic as Jest tests that could run independently of a live server.

Integration testing used a Postman collection maintained throughout development. Every new endpoint got corresponding tests covering at minimum: valid input, missing required field, and invalid authentication token.

User acceptance testing involved five people unfamiliar with the system asked to complete a booking, reschedule it, and cancel it. Two of the five initially clicked the provider registration link instead of the customer path – they didn't realise the platform served two different user types from the same landing page. We hadn't anticipated that confusion at all; internally we always navigated directly to the routes we needed. That led to a landing page redesign with clearly separated entry points. The other major finding was the date picker issue documented in NFR-04 – one user spent over a minute fighting a control that resisted their expected interaction pattern. Both issues were fixed before the final submission.

5.6 Tools and Environment

We used MongoDB Compass specifically because Jainil found it significantly easier than the command line when inspecting the appointments collection during debugging. When a slot query was returning unexpected results, being able to browse documents visually and run queries without remembering the exact syntax saved time. Not every team member uses a GUI for database work – Daksh mostly used the Atlas web interface – but Compass was the right tool for that particular debugging workflow.

Table 5.1: Tools Used During Development

Category	Tool	Purpose
Version Control	Git + GitHub	Source control, team collaboration
IDE	VS Code	Primary development environment
API Testing	Postman	Endpoint testing and collection management
UI Prototyping	Figma	Wireframes and layout planning
Database GUI	MongoDB Compass	Schema inspection and visual debugging
Frontend Deploy	Vercel	Continuous deployment from GitHub
Backend Deploy	Render	Node.js cloud hosting
DB Hosting	MongoDB Atlas	Managed cloud database

CHAPTER 6

IMPLEMENTATION

The implementation diverged from the design in at least four places that mattered. The slot overlap bug changed how conflict checking was written. The debounce value on the slot fetch required empirical tuning. The frontend state management was refactored midway through. And the `/utils` folder wasn't in the original architecture at all. This chapter describes both what was planned and what got built, because the differences between the two are where the learning happened.

6.1 Backend Implementation

6.1.1 Project Structure

The Node.js backend uses a controller-model-route pattern:

/backend		
	/controllers	business logic per resource
	/models	Mongoose schemas
	/routes	Express route definitions
	/middleware	auth, role check, error handler
	/utils	email sender, slot generator
`	server.js	entry point, DB connection

The `/utils` folder wasn't in the original plan. It was added in week three after Daksh caught a code review issue where both the email sender and the slot generator function had been copy-pasted into three separate controllers. Each copy had drifted slightly – different error handling, one with a `console.log` that got removed in another. He refactored both into shared utility functions, which saved at least two debugging sessions later when a change to the slot generator needed to propagate everywhere.

6.1.2 Slot Availability Logic

When a customer requests available slots for a specific date and service, the backend:

1. Retrieves the provider's working hours for that day of the week, including split-shift configuration.
2. Generates all possible start times based on service duration (e.g., 9am–5pm at 45-minute intervals produces 9:00, 9:45, 10:30, and so on).
3. Queries appointments for all confirmed bookings on that date for the selected staff member.
4. Removes any generated slot that overlaps with an existing booking or a configured break period.
5. Returns only the remaining free slots.

The overlap check in step 4 is where we had the most trouble. The first version used exact start-time matching – if an existing booking started at 3:00 PM, it would block 3:00 PM for new bookings. But a 60-minute service booked at 3:00 PM should also block 3:30 PM for a 30-minute service, because the first service is still running. The first version didn't catch that partial-overlap case. Aksh found it by writing a test specifically for the scenario: 60-minute booking at 3:00 PM, attempt to book a 30-minute slot at 3:30 PM. The test failed, confirming the suspicion. The fix was switching from equality comparison to interval overlap detection: a new slot at time t with duration d is blocked if any existing booking b satisfies $b.start < t + d$ and $b.end > t$. That one line of logic change is the reason the system works correctly.

The entire check happens at query time, not on a schedule, which means displayed slots are always current.

6.1.3 Authentication Middleware

JWT verification runs as Express middleware before any protected route handler. It extracts the token from the `Authorization: Bearer` header, verifies the signature, and attaches the decoded user to `req.user`. Missing, expired, or tampered tokens return a 401 – the route handler never runs.

A separate role-check middleware verifies `req.user.role === 'provider'` for admin routes, running as a second step in the chain after authentication rather than folded into it. This keeps the two concerns testable independently.

6.1.4 Email Notification Service

Two email types are sent. Confirmation emails trigger immediately when an appointment is created. Reminder emails are sent by a `node-cron` job that runs every hour and queries for appointments where `dateTime` is between `now + 23h` and `now + 25h` and `reminderSent === false`. For each match, the email goes out and `reminderSent` is set to `true` to prevent duplicates.

The two-hour window (23–25 hours rather than exactly 24) is intentional: an hourly job running against a 24-hour exact window would miss appointments that land in the gap depending on when they were created. In the four weeks of testing we ran, no appointment missed its reminder window, though a gap exactly at the hourly boundary would be difficult to detect through normal testing – it would require an appointment created at precisely the wrong minute. We acknowledge this as a theoretical gap rather than an observed one.

6.2 Frontend Implementation

6.2.1 Project Structure

/frontend/src		
	/pages	Home, Booking, Dashboard, Auth
	/components	SlotPicker, BookingCard, Navbar
	/context	AuthContext, BookingContext
	/hooks	useAuth, useFetchSlots
`	/services	API call wrappers using Axios

6.2.2 Real-Time Slot Display

The slot picker fetches available slots from the API whenever the user changes the selected date or staff member. The fetch is debounced by 300ms. We arrived at that specific value after Om noticed during local testing that changing the date three times quickly was firing six API calls visible in the browser network tab, and occasionally a slower earlier response would arrive after a faster later one – overwriting the correct result with stale data. The first attempt used a 500ms debounce, which felt too sluggish when clicking between dates quickly. 300ms was the lowest value that eliminated the

race condition while keeping the UI feeling immediate.

6.2.3 State Management and the Prop Drilling Refactor

The booking flow state – the selected appointment details needed across the booking confirmation, reschedule, and cancellation screens – started life being passed through props from the top-level `BookingPage` component down through `BookingFlow`, `SlotConfirmCard`, and eventually `RescheduleButton`. Four levels of prop drilling. It worked until Om tried to make the reschedule button in `SlotConfirmCard` reflect the updated appointment time after a reschedule had been confirmed. The updated state lived in `BookingPage`, but `SlotConfirmCard` only received a snapshot of it at render time. He spent twenty minutes tracing why a change in one place wasn't reflected somewhere three levels down before deciding the structure was the problem, not the logic.

The refactor to React Context took about an hour. `BookingContext` now holds the shared appointment state and exposes it to any component in the tree without explicit prop passing. The component code got noticeably cleaner, and the reschedule flow that had been the immediate problem became straightforward to add.

6.3 Implementation Status

Table 6.1: Feature Implementation Status

Feature	Status	Phase
User registration and login	Done	Sem 6
Real-time slot booking	Done	Sem 6
Email confirmation	Done	Sem 6
Email reminder (24hr)	Done	Sem 6
Provider dashboard – Schedule	Done	Sem 6
Provider dashboard – Analytics	Partial	Sem 6
Calendar sync (iCal export)	In progress	Sem 6/7
Payment integration	Deferred	Sem 7
Mobile application	Deferred	Sem 7
AI slot recommendations	Deferred	Sem 7

CHAPTER 7

CONCLUSION

7.1 Summary of Work Done

When we started this project, the problem felt straightforward. Build a booking system. Turns out the problem is straightforward – the implementation is where it gets complicated. Getting the slot logic right took longer than expected. The partial-overlap bug – a 30-minute slot showing as available at 3:30 PM when a 60-minute service had started at 3:00 PM – would have made it into production without a test suite that covered interval overlap explicitly rather than just same-start-time conflicts.

By the end of this semester, Slotify is a working web application covering the core use case end-to-end: a customer can register, find a provider, view real-time available slots, book one, receive an email confirmation, and get a reminder the day before. A registered business can configure services and staff, manage bookings from a dashboard, and see basic analytics about their operation.

7.2 Objectives Met

Table 7.1 maps where we actually landed. The “In Progress” and “Partially Done” labels tell a more honest story than a clean checkmark list would.

Table 7.1: Objective Completion Status

#	Objective	Status
1	Customer-facing booking with real-time slot display	Completed
2	Provider dashboard with schedule management and double-booking prevention	Completed
3	Automated email confirmation and 24-hour reminder	Completed
4	Calendar synchronisation and cloud deployment	Partial / Completed
5	Business analytics module	Partially done

7.3 Challenges Encountered

The slot overlap bug was the most technically frustrating moment in the project. We had tested booking, rescheduling, and cancellation for two weeks without finding it because

all our test cases used non-overlapping appointment times. Writing a test specifically for a partial-overlap scenario – a 60-minute booking at 3:00 PM followed by an attempt to book 3:30 PM – is not something you do unless you’re suspicious the logic might be too simple. Aksh was suspicious, ran the test, and it failed. The fix took twenty minutes. The debugging time before the test existed was considerably longer.

The prop drilling refactor described in Chapter 6 was a different kind of frustration – not a bug, but a design decision that worked at small scale and became obviously wrong as the component tree grew. The point at which Om decided to refactor was when he spent twenty minutes tracing shared booking state through `BookingPage` down to `BookingFlow` down to `SlotConfirmCard` trying to understand why a reschedule wasn’t reflected in the UI. React Context fixed it quickly, which was also slightly annoying – the time spent working around the wrong structure before the refactor wasn’t recoverable.

7.4 Key Takeaways

The lesson that caught us most off guard was how much more useful the two user conversations were compared to the literature review. We went into the project expecting the research to shape the product and the users to validate it. What actually happened was closer to the opposite – the papers confirmed decisions we’d already made from intuition (write tests, choose MongoDB, use REST), while the salon owner’s observation about dead slots changed a requirement we hadn’t known we were missing. The cancellation-recovery feature, the immediate slot release on cancellation, the notification log for debugging – all of these came from real conversations, not from papers.

The other concrete takeaway: writing tests before declaring code done saves more time than it costs. The slot overlap bug is the clearest example, but it wasn’t the only one. Having a Postman collection that ran against every endpoint meant that when Daksh changed the slot-generation logic, we knew within a minute whether anything else had broken.

MongoDB was the right call, and the experience of starting in PostgreSQL and switching mid-design made that clear in a way that reading about it in Sadalage and Fowler [9] wouldn’t have.

CHAPTER 8

FUTURE WORK

8.1 What Comes Next

The work done this semester is a foundation, not a finished product. The items below are not given equal weight intentionally – some are more commercially critical than others, and the order here reflects that rather than treating everything as equally interesting.

8.2 Payment Integration

This is the most commercially important item and deserves more space than the others. When a customer prepays even a small deposit at the time of booking, no-show rates drop – the mechanism is that financial commitment raises the perceived cost of cancellation. This is supported by behavioural economics literature on commitment devices, and is consistent with the pattern documented in [6] – the mechanism is simple: financial commitment makes last-minute cancellation feel like it has a cost. The salon owner we spoke to estimated she loses around 800–1000 rupees per week to empty slots from no-shows. A refundable deposit of 100 rupees would change that calculus for most customers without feeling punitive.

The plan is to integrate Razorpay for the Indian market and Stripe for broader use. Each provider would configure their own policy: full payment at booking, partial deposit, or no prepayment. The refund flow is where most of the engineering complexity lives – it requires handling partial refunds, cancellation windows (full refund if cancelled 24 hours in advance, no refund within 4 hours), and edge cases like provider cancellations where the customer should always get a full refund regardless. The payment integration is what all of this hangs on, which is why it comes first in priority even though the AI recommendations feature sounds more technically interesting.

8.3 Mobile Application

Most customers booking a salon or clinic appointment are on their phones. The web app is responsive, but a native experience for something as interaction-heavy as date selection and slot picking is noticeably better than a mobile browser. The plan is a React

Native application [14] for Android and iOS that shares the existing backend API – no new server code needed. This is exactly the scenario the three-tier architecture from Chapter 4 was designed for.

8.4 AI-Based Slot Recommendations

This is the most technically ambitious item and the most uncertain. The basic idea is using historical booking patterns to surface optimal slots – if most customers who book haircuts on weekdays choose 10am–12pm windows, show those first.

The hard problem is cold start. A new business on Slotify has zero historical bookings to learn from. The recommendation engine needs a fallback: suggest slots that are most popular across all similar businesses in the same category. We don't know yet at what booking volume the personalised model starts to meaningfully outperform that heuristic. That question needs an answer before this feature is worth building, which is part of why it's deferred. We're not building it until we have enough real booking data to test whether personalisation adds anything.

8.5 Advanced Analytics

The current analytics tab shows bookings per day and three summary numbers. The harder problem here is not the queries – the data is already being stored in a shape that would support richer analysis. It's deciding what a salon owner actually wants to see versus what we as developers think is analytically interesting.

When we asked the salon owner this question directly, her answer was immediate: she wanted to know which of her staff members was getting the most repeat bookings. Not total bookings – repeat bookings specifically. That's a very different query from the time-series chart of daily volume that our developer instinct would default to, and it requires tracking customer-to-staff relationship data that we're not currently surfacing anywhere in the UI. Building analytics that are actually useful means answering her question first, not the question we would have thought to ask.

8.6 Multi-Language Support

Gujarati and Hindi support is achievable quickly given the frontend already uses React components with isolated text strings – adding i18next and working with a native speaker to review the copy would cover most of the surface area. This is a one-sprint task rather

than a major feature. Low-hanging fruit for reaching more of the actual target market.

8.7 Multi-Branch Business Support

Some businesses operate across multiple locations. In Vadodara specifically, chains like Looks Salon – which has several branches across the city – represent exactly the kind of operator this feature would serve. The current data model structurally supports it (each business is a separate entity), but the provider interface has no way to manage multiple branches under one login. Adding a parent-business relationship to the data model and a multi-branch view to the dashboard is on the roadmap for Semester 7 alongside the payment integration.

References

- [1] Bailey, N.T.J. and Welch, J.D., 1952. Appointment systems in hospital outpatient departments. *The Lancet*, 259(6718), pp. 1105–1108.
- [2] Cayirli, T. and Veral, E., 2003. Outpatient scheduling in health care: a review of literature. *Production and Operations Management*, 12(4), pp. 519–549.
- [3] Fielding, R.T., 2000. *Architectural Styles and the Design of Network-Based Software Architectures*. Doctoral Dissertation, University of California, Irvine.
- [4] Gupta, D. and Denton, B., 2008. Appointment scheduling in health care: challenges and opportunities. *IIE Transactions*, 40(9), pp. 800–819.
- [5] Madden, N., 2020. *API Security in Action*. Manning Publications, Shelter Island, NY.
- [6] Parikh, A., Gupta, M., Wilson, A., Fields, K., Cosgrove, N. and Kostis, J., 2010. The effectiveness of outpatient appointment reminder systems in reducing no-show rates. *The American Journal of Medicine*, 123(6), pp. 542–548.
- [7] Pressman, R.S. and Maxim, B.R., 2014. *Software Engineering: A Practitioner's Approach*. 8th ed. McGraw-Hill Education, New York.
- [8] Richardson, L. and Ruby, S., 2007. *RESTful Web Services*. O'Reilly Media, Sebastopol, CA.
- [9] Sadalage, P.J. and Fowler, M., 2012. *NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence*. Addison-Wesley Professional, Upper Saddle River, NJ.
- [10] Shmueli, A., Hadad, N. and Haklai, Z., 2003. Ambulatory care utilization: wait time, type of visit and medical condition. *Israel Medical Association Journal*, 5(2), pp. 114–118.
- [11] Sommerville, I., 2016. *Software Engineering*. 10th ed. Pearson Education Limited, Harlow.

- [12] Stack Overflow, 2023. *Stack Overflow Developer Survey 2023* [Online]. Available at: <https://survey.stackoverflow.co/2023> [Accessed: 10 February 2025].
- [13] MongoDB Inc., 2024. *MongoDB Documentation: Data Modeling* [Online]. Available at: <https://www.mongodb.com/docs/manual/data-modeling/> [Accessed: 15 January 2025].
- [14] React.js Team, 2024. *React: A JavaScript Library for Building User Interfaces* [Online]. Available at: <https://react.dev> [Accessed: 5 January 2025].
- [15] Nodemailer Project, 2024. *Nodemailer – Send Emails from Node.js* [Online]. Available at: <https://nodemailer.com/about/> [Accessed: 8 February 2025].
- [16] OpenJS Foundation, 2024. *Express.js – Fast, Minimalist Web Framework for Node.js* [Online]. Available at: <https://expressjs.com> [Accessed: 6 January 2025].
- [17] Auth0 Inc., 2024. *Introduction to JSON Web Tokens* [Online]. Available at: <https://jwt.io/introduction> [Accessed: 12 February 2025].
- [18] Tailwind Labs, 2024. *Tailwind CSS Documentation* [Online]. Available at: <https://tailwindcss.com/docs> [Accessed: 11 January 2025].
- [19] Vercel Inc., 2024. *Vercel Documentation* [Online]. Available at: <https://vercel.com/docs> [Accessed: 20 January 2025].
- [20] Cloudinary Inc., 2024. *Cloudinary Developer Documentation* [Online]. Available at: <https://cloudinary.com/documentation> [Accessed: 10 January 2025].

APPENDIX A PLAGIARISM CERTIFICATE

PLAGIARISM CHECK CERTIFICATE

As per Parul University's Zero Tolerance Policy against Plagiarism, the plagiarism check for this project report has been carried out using open-source plagiarism detection software. The generated certificate is attached below.

Project Title	Face Detection and Recognition System
Students	AKSH THAKKAR, OM MAURYA, DAKSH PATEL, JAINIL PATEL
Software Used	
Similarity (%)	
Date	

Signature of Student(s)

Signature of Guide

Dr. Sneha Soni

APPENDIX SELF-DECLARATION

SELF-DECLARATION

(by Student/s & Supervisor)

We, the undersigned students of the 6th Semester, Computer Engineering Department, Parul Institute of Engineering and Technology (Parul University), hereby declare that:

1. The project entitled “**Slotify: A Smart Appointment Booking System for Service-Based Businesses**” has been carried out by us under the guidance of **Dr. Sneha Soni**.
2. The work is original and has not been submitted elsewhere for any degree or diploma.
3. We have not purchased solutions developed by any third party.
4. We understand that if the above declaration is found to be untrue, it may result in cancellation of the project and failure in the subject.

Sr.	Name of Student	Enrollment No.	Signature
1	Aksh Thakkar	2403031057002	
2	Om Maurya	2403031057048	
3	Daksh Patel	2043031057065	
4	Jainil Patel	2403031057071	

Date: _____

Place: Limda, Vaghodia, Vadodara

Signature of Supervisor

Dr. Sneha Soni

Signature of HOD

Dr. Shailendra K Mishra